

1. Write a shell script to print arguments passed to the script in reverse order.

Input: sh script.sh 2 5 3

Output:

3

5

2

Ans:

-> vi 1.reverse-order.sh

```
#!/bin/bash
```

```
for ((i=$#; i>0; i--))
```

```
do
```

```
    echo "${!i}"
```

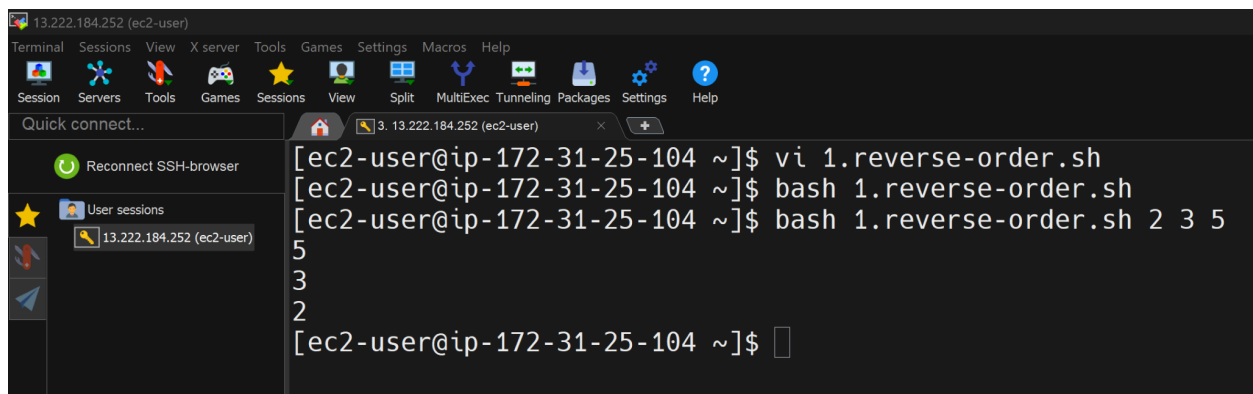
```
done
```

Output: [ec2-user@ip-172-31-39-119 ~]\$ bash 1.reverse-order.sh 2 3 5

5

3

2

A screenshot of a terminal window with a dark background. The window title is "13.222.184.252 (ec2-user)". The menu bar includes Terminal, Sessions, View, X server, Tools, Games, Settings, Macros, and Help. The toolbar shows icons for Session, Servers, Tools, Games, Sessions, View, Split, MultiExec, Tunneling, Packages, Settings, and Help. The left sidebar has a "Quick connect..." section with a "Reconnect SSH-browser" button and a "User sessions" section showing a session for "13.222.184.252 (ec2-user)". The main terminal area shows the following commands and output:

```
[ec2-user@ip-172-31-25-104 ~]$ vi 1.reverse-order.sh
[ec2-user@ip-172-31-25-104 ~]$ bash 1.reverse-order.sh
[ec2-user@ip-172-31-25-104 ~]$ bash 1.reverse-order.sh 2 3 5
5
3
2
[ec2-user@ip-172-31-25-104 ~]$
```

Explanation for me: Why "\${!i}" ?

This is called indirect parameter expansion in Bash.

Normal case:

```
echo "$1"
```

This prints the first argument.

But here i is a variable:

```
i=3
```

If you do:

```
echo "$i"
```

It prints 3

But you want:

```
echo "$3"
```

So how do you dynamically access \$1, \$2, \$3 using a variable?

That's where \${!i} comes in.

2. Write a script that monitors system performance and sends a mail notification with warning messages about resource usage like ram, disk usage.

Ans:

First we set up an email client on the server to send email from the server.

```
-> sudo yum install s-nail cyrus-sasl-plain -y
```

Note: First you need to get the app password from gmail for smtp settings. Go to manage your google account >> search for "App password" from the search bar as it won't be visible in the dashboard. Then you can set the app password and use it.

```
→ vi ~/.mailrc
```

```
# Set MTA to use Gmail's SMTP server with STARTTLS
```

```
set mta=smtp://smtp.gmail.com:587
```

```
# Authentication settings
```

```
set smtp-auth=login
```

```
set smtp-auth-user=nagarajkamath602@gmail.com
```

```
#Update gmail app password in smtp-auth-password
set smtp-auth-password=wcelrtsjlqbenfzs
# Email settings
set from="nagarajkamath602@gmail.com"
# Enable TLS
set ssl-verify=ignore
set smtp-use-starttls
```

→ :wq

Once smtp is set up, we will write a script to monitor resource usage.

-> vi 2.performance_report.sh

```
#!/bin/bash
```

```
DISK_THRESHOLD=80
#For testing, I set RAM threshold as 25
RAM_THRESHOLD=25
EMAIL="nagarajkamath602@gmail.com"
```

```
# Get usage values
disk_usage=$(df -h / | awk 'NR==2 {print $5}' | sed 's/%//')
ram_usage=$(free | awk 'NR==2 {print int($3*100/$2)}')
load_avg=$(uptime | awk -F'load average:' '{print $2}')
```

```
alert_flag=0
```

```
# Check Disk
if [ "$disk_usage" -ge "$DISK_THRESHOLD" ]; then
    disk_status="HIGH"
    alert_flag=1
else
```

```

        disk_status="OK"
    fi

    # Check RAM
    if [ "$ram_usage" -ge "$RAM_THRESHOLD" ]; then
        ram_status="HIGH"
        alert_flag=1
    else
        ram_status="OK"
    fi

    # Send email only if any resource is HIGH
    if [ "$alert_flag" -eq 1 ]; then
        echo -e "
=====
                SYSTEM RESOURCE ALERT
=====

Hostname : $(hostname)
IP       : $(hostname -I | awk '{print $1}')
Date     : $(date)

-----
Resource Usage
-----

Disk Usage : ${disk_usage}% [$disk_status]
RAM Usage  : ${ram_usage}%  [$ram_status]

Load Average (1m, 5m, 15m):
$load_avg

-----
Action Required: Please investigate.

```

```
=====
" | mail -s "ALERT: Resource Usage on $(hostname)" "$EMAIL"
fi
```

```
:wq
```

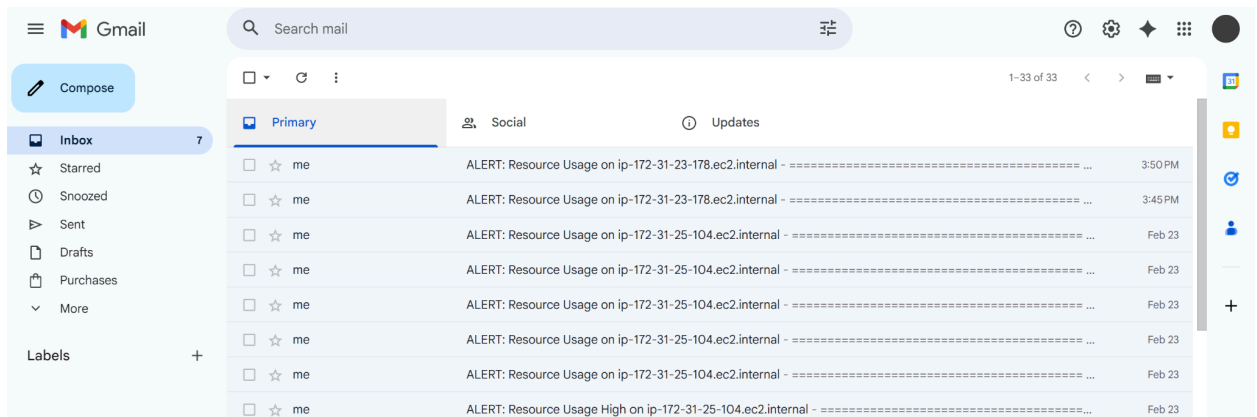
```
-> crontab -e
```

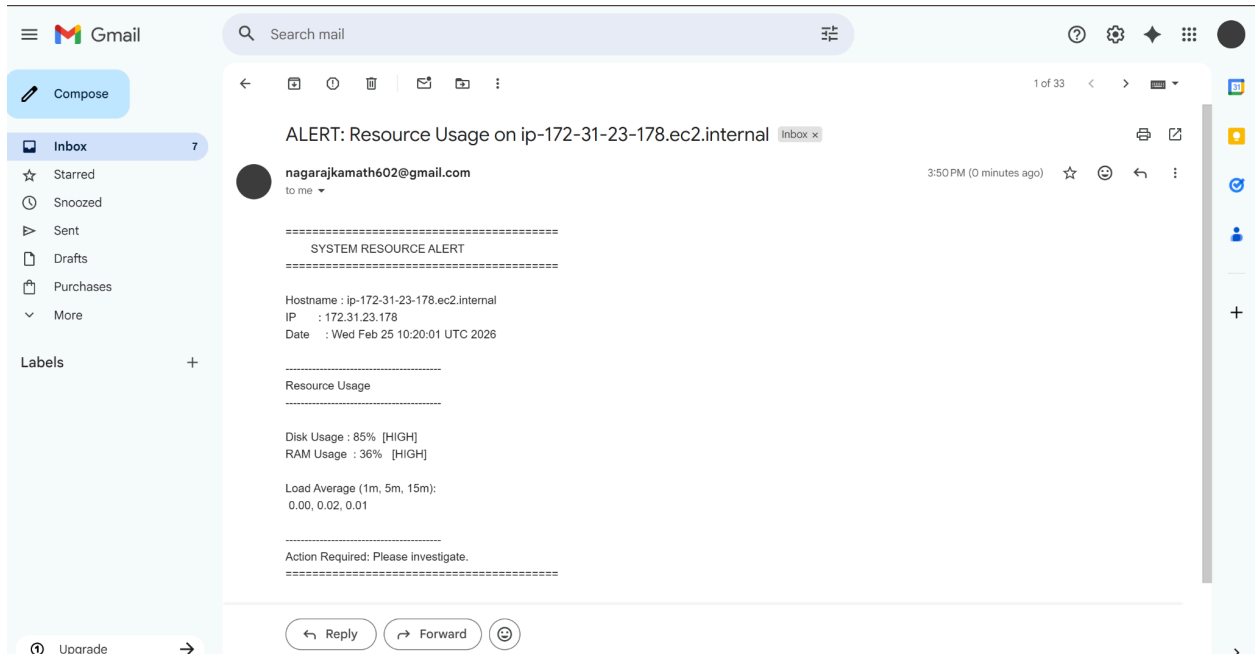
```
* /5 * * * * bash /home/ec2-user/2.performance_report.sh - -> run it every 5
minutes
```

For testing purpose: To generate high disk usage run:
fallocate -l 7G testfile.img

To kill it:

```
rm -f testfile.img
```





3. Write a script to find the files in a directory that were last modified more than 4 weeks ago [For Practise use Mins/Days] and delete them, starting with the ones that consume the most space.

Ans:

-> vi cleanup_old_files.sh

```
#!/bin/bash
```

```
DIR="/home/ec2-user/testdir"
```

```
find "$DIR" -type f -mtime +28 -size +10M -exec echo "Deleting: {}" \; -delete
```

Output: Created file with 10mb storage 40 days ago. (Used wget to get tomcat file and copied it to different files and used touch -d "40 days ago" <file-name> to change the timestamp)

→ Bash cleanup_old_files.sh

```
[ec2-user@ip-172-31-23-178 ~]$ bash cleanup_old_files.sh
Deleting: /home/ec2-user/testdir/h1
Deleting: /home/ec2-user/testdir/n1
Deleting: /home/ec2-user/testdir/g12
Deleting: /home/ec2-user/testdir/d71
[ec2-user@ip-172-31-23-178 ~]$
```

4. Explain the difference between running a script with '`./script`' and running it with '`nohup ./script &`'

Ans:

- `./script` → runs the script in the foreground, tied to the terminal.
- `nohup ./script &` → runs the script in the background, independent of the terminal.

Explanation for me:

1. Running a script with `./script`

Let's say you have a script called `backup_data.sh` that takes a few minutes to run. If you run it with `./backup_data.sh`, the script will occupy the terminal until it finishes:

```
$ ./backup_data.sh
Backing up data... (takes 5 minutes)
```

You won't be able to use the terminal until the script finishes. If you close the terminal or press `Ctrl+C`, the script will be interrupted.

2. Running a script with `nohup ./script &`

If you run the same script with `nohup ./backup_data.sh &`, the script will run in the background, freeing up the terminal:

```
$ nohup ./backup_data.sh &
```

```
[1] 12345
```

```
$
```

The script will continue running even if you close the terminal or press Ctrl+C.

The output will be logged to a file named nohup.out:

```
$ cat nohup.out
```

```
Backing up data... (takes 5 minutes)
```

You can check the script's progress by tailing the log file:

```
$ tail -f nohup.out
```

These examples illustrate the difference between running a script in the foreground (with ./script) and running it in the background (with nohup ./script &).

5. Read an integer ' n ' and generate the following pattern:

1

1 2

1 2 3

1 2 3 4 up to ' n ' rows.

Ans:

-> vi pattern.sh

```
#!/bin/bash
```

```
echo "Enter the number"
```

```
read n
```

```
for((i=1; i<=n; i++))
```

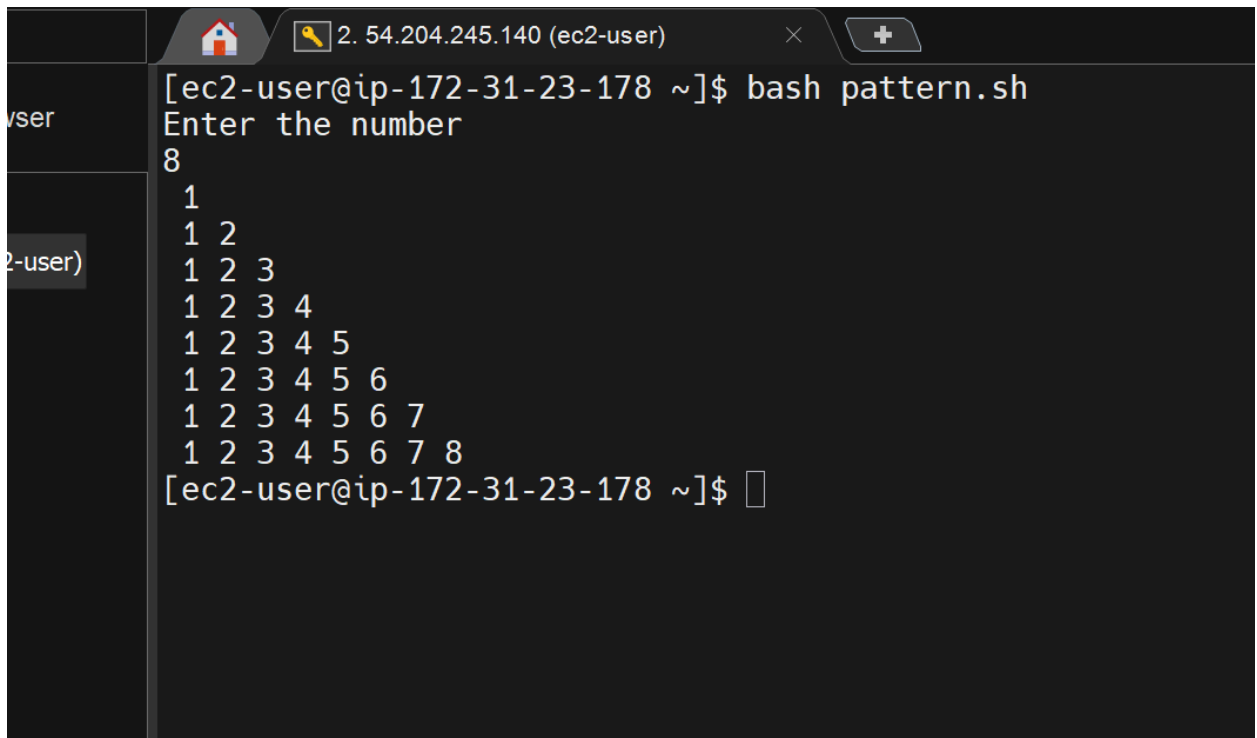
```
do
```



```

        for((j=1; j<=i; j++))
do
    echo -n " $j"
done
    echo ""
done

```



```

[ec2-user@ip-172-31-23-178 ~]$ bash pattern.sh
Enter the number
8
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
[ec2-user@ip-172-31-23-178 ~]$

```

6. Write a script to display the contents of the file in reverse order without using tac command

Ans:

-> vi reverse_file.sh

```
#!/bin/bash
```

```
while read line
```

```
do
```

```
    output="$line"$'\n'"$output"
```

```
done < "$1"
```

```
echo -e "$output"
```

Output:

```
-> vi example.txt
```

```
100
```

```
200
```

```
300
```

```
400
```

```
500
```

```
600
```

```
700
```

```
800
```

```
900
```

```
:wq
```

```
-> bash reverse_file.sh example.txt
```

```
900
```

```
800
```

```
700
```

```
600
```

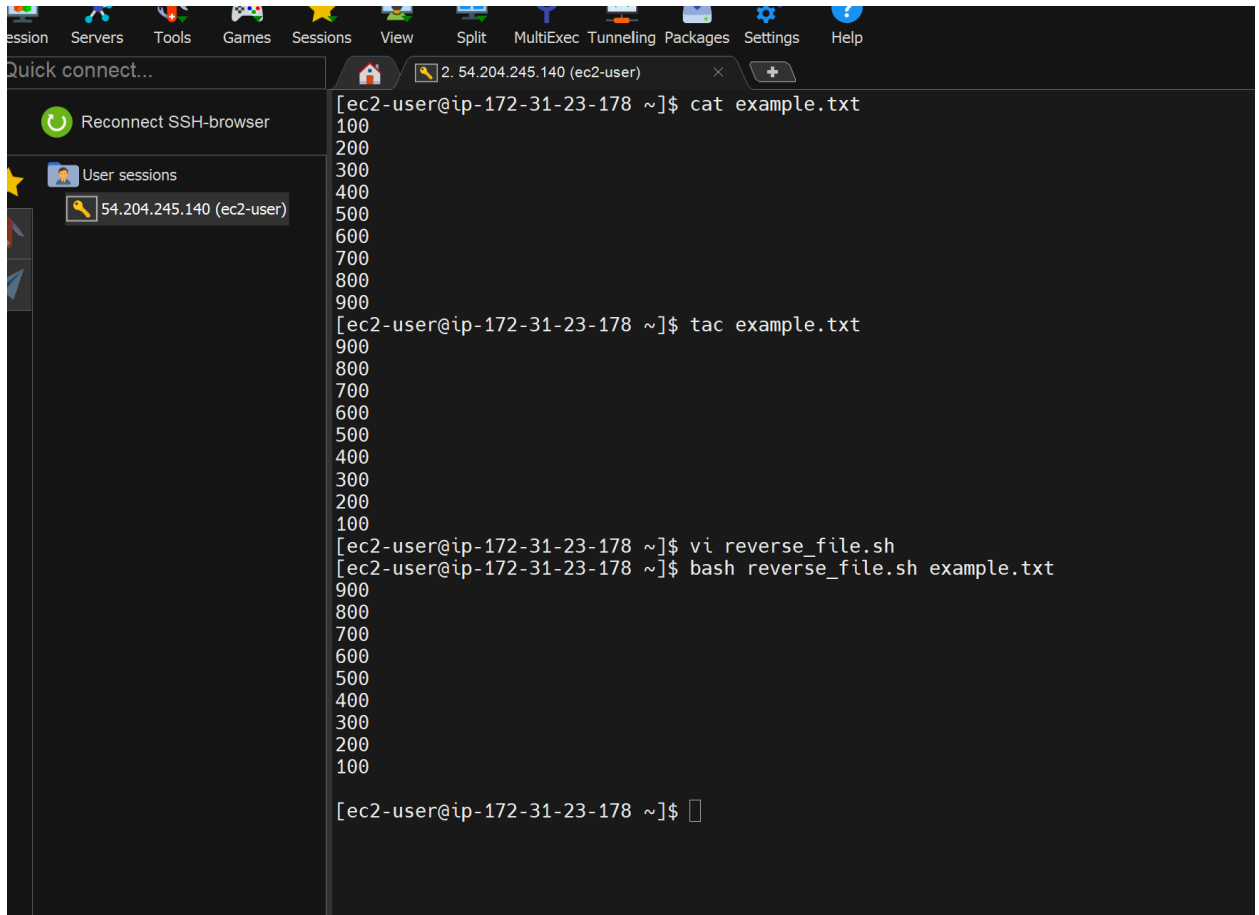
```
500
```

```
400
```

```
300
```

```
200
```

```
100
```



```
[ec2-user@ip-172-31-23-178 ~]$ cat example.txt
100
200
300
400
500
600
700
800
900
[ec2-user@ip-172-31-23-178 ~]$ tac example.txt
900
800
700
600
500
400
300
200
100
[ec2-user@ip-172-31-23-178 ~]$ vi reverse_file.sh
[ec2-user@ip-172-31-23-178 ~]$ bash reverse_file.sh example.txt
900
800
700
600
500
400
300
200
100
[ec2-user@ip-172-31-23-178 ~]$
```

7. Write a script to reverse a string using while loop

Input: hello

Output: olleh

Ans:

-> vi reverse_string.sh

```
#!/bin/bash
```

```
echo "Enter a string:"
```

```
read string
```

```
i=$(echo -n "$string" | wc -c)
```

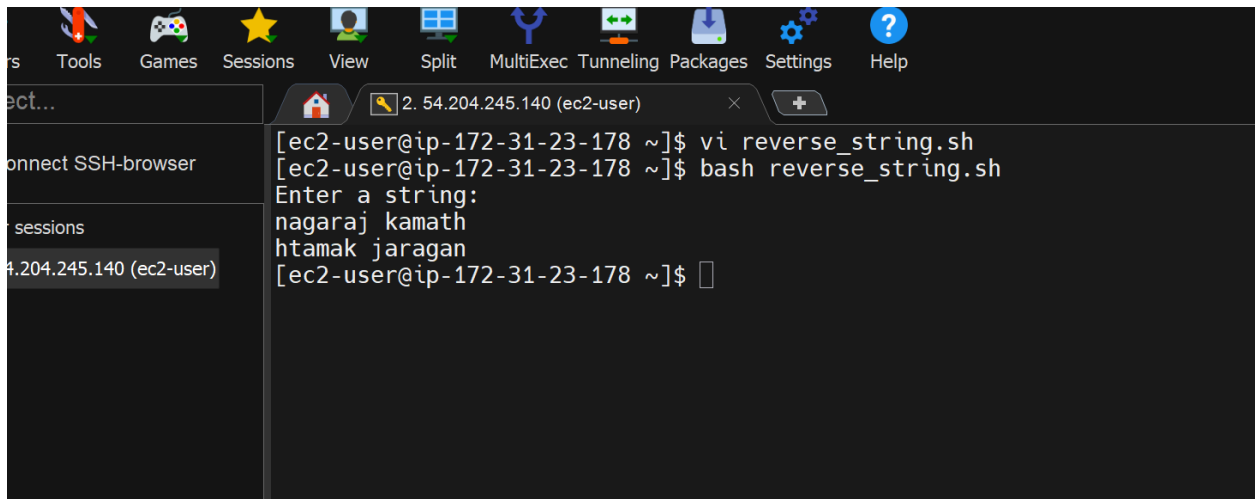
```
while [ $i -gt 0 ]
```

```
do
```

```
char=$(echo -n "$string" | cut -c $i)
echo -n "$char"
((i--))
done
```

echo

Output:



```
[ec2-user@ip-172-31-23-178 ~]$ vi reverse_string.sh
[ec2-user@ip-172-31-23-178 ~]$ bash reverse_string.sh
Enter a string:
nagaraj kamath
htamak jaragan
[ec2-user@ip-172-31-23-178 ~]$
```

8. Read an integer 'n' and generate the following pattern:

```
1
2 3
4 5 6
7 8 9 10
up to n rows.
```

Ans:

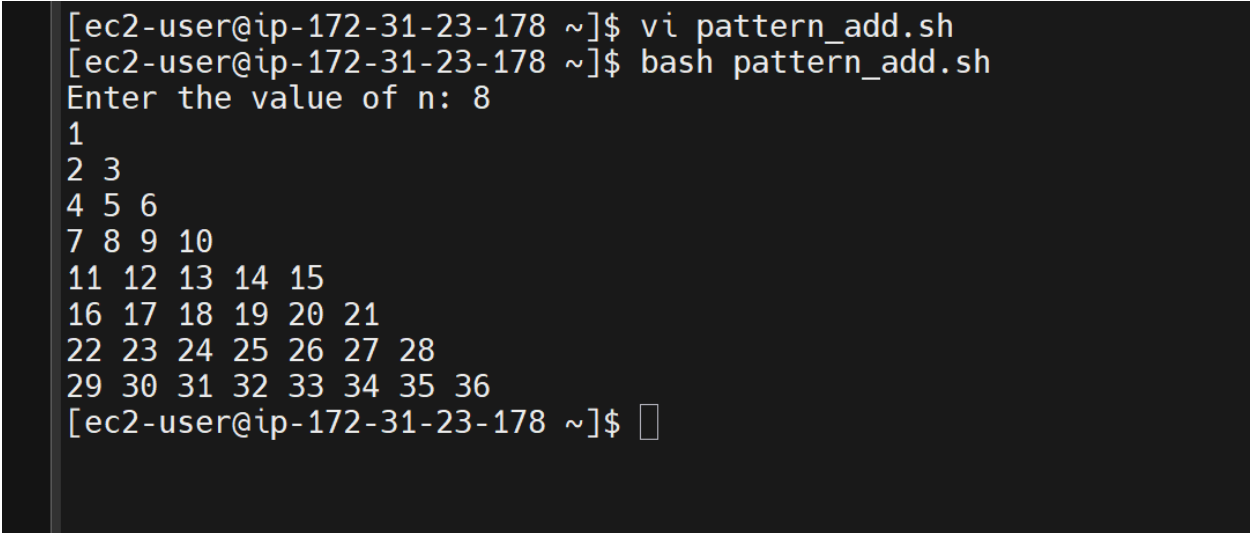
-> vi pattern_add.sh

```
#!/bin/bash
```

```
read -p "Enter the value of n: " n
```

```
num=1
for ((i = 1; i <= n; i++)); do
    for ((j = 1; j <= i; j++)); do
        echo -n "$num "
        num=$((num + 1))
    done
done
echo
```

Output:



```
[ec2-user@ip-172-31-23-178 ~]$ vi pattern_add.sh
[ec2-user@ip-172-31-23-178 ~]$ bash pattern_add.sh
Enter the value of n: 8
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31 32 33 34 35 36
[ec2-user@ip-172-31-23-178 ~]$
```

9. Write a script that rename a file or directory by replacing its letters with lowercase from uppercase letters. Example: TEst.txt → test.txt

Ans:

-> vi rename.sh

```
#!/bin/bash
```

```
echo "Enter the file or directory name"
read name
```

```
echo -e "\n\n$name is renamed to:"
```

```
new_name=${name,,}
```

```
# Rename the file or directory
```

```
mv "$name" "$new_name"
```

```
echo -e " $new_name\n"
```

Explanation: The `${name,,}` syntax converts the name to lowercase, using the bash parameter expansion feature.

Output:

```
[ec2-user@ip-172-31-23-178 ~]$ mkdir Dir
[ec2-user@ip-172-31-23-178 ~]$ touch Ghe12
[ec2-user@ip-172-31-23-178 ~]$ bash rename.sh
Enter the file or directory name
Dir

Dir is renamed to:
dir

[ec2-user@ip-172-31-23-178 ~]$ ls
1.reverse-order.sh  apache-tomcat-11.0.18.tar.gz  dead.letter  pattern.sh  reverse_file.sh  testfile.img
2.performance_report.sh  arguments_check.sh  dir  pattern_add.sh  reverse_string.sh
Ghe12  cleanup_old_files.sh  example.txt  rename.sh  testdir
[ec2-user@ip-172-31-23-178 ~]$ bash rename.sh
Enter the file or directory name
Ghe12

Ghe12 is renamed to:
ghe12

[ec2-user@ip-172-31-23-178 ~]$
```

10. Write a script that takes multiple inputs from the user as arguments and only prints the arguments if they are integers

Input: `sh script.sh abc 2 t5 4 efg ghi 34 67 1 2`

Output:

The integer arguments are:

2

4

34

67

1

2

Ans:

-> vi arguments_check.sh

```
#!/bin/bash
```

```
echo "The integer arguments are:"
```

```
for arg in "$@"  
do  
    if [[ "$arg" =~ ^[0-9]+$ ]]  
    then  
        echo "$arg"  
    fi  
done
```

Explanation: Using =~ means

```
[[ "$a" =~ ^[0-9]+$ ]]
```

Matches full regular expression.

^ Start of string

[0-9]+ One or more digits

\$ End of string

Output:

```
Games Sessions View Split MultiExec Tunneling Packages Settings Help
2. 54.204.245.140 (ec2-user) x +
browser [ec2-user@ip-172-31-23-178 ~]$ vi reverse_string.sh
[ec2-user@ip-172-31-23-178 ~]$ bash reverse_string.sh
Enter a string:
nagaraj .kamath
htamak jaragan
(ec2-user) [ec2-user@ip-172-31-23-178 ~]$ vi arguments_check.sh
[ec2-user@ip-172-31-23-178 ~]$ bash arguments_check.sh abc 2 t5 4 efg ghi 34 67 1 2
The integer arguments are:
2
4
34
67
1
2
[ec2-user@ip-172-31-23-178 ~]$
```